

Docker & PostgreSQL Container

SWEN1 | BIF3

Table of Contents

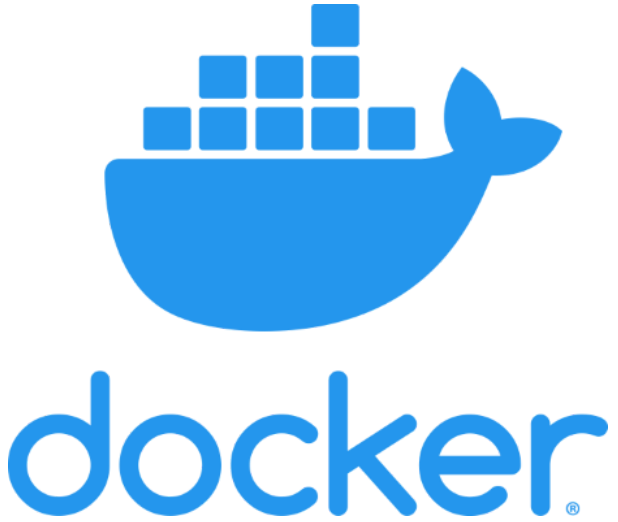
- Containerization Fundamentals
- Docker Architecture
- Running PostgreSQL Containers
- Container Management Basics
- Troubleshooting Common Issues



Containerization Fundamentals

What is Docker?

- **Lightweight virtualization** - Isolated environments for applications
- **Package once, run anywhere** - Consistent environments from dev to production
- **Resource efficient** - Shares host OS kernel (unlike VMs)
- **Industry standard** - Used by over 90% of Fortune 100 companies according to Docker's 2024 ecosystem report



Key components:

- **Images** - Read-only templates (e.g., PostgreSQL image)
- **Containers** - Running instances of images
- **Docker Engine** - Runtime that manages containers

Why Containers for Databases?



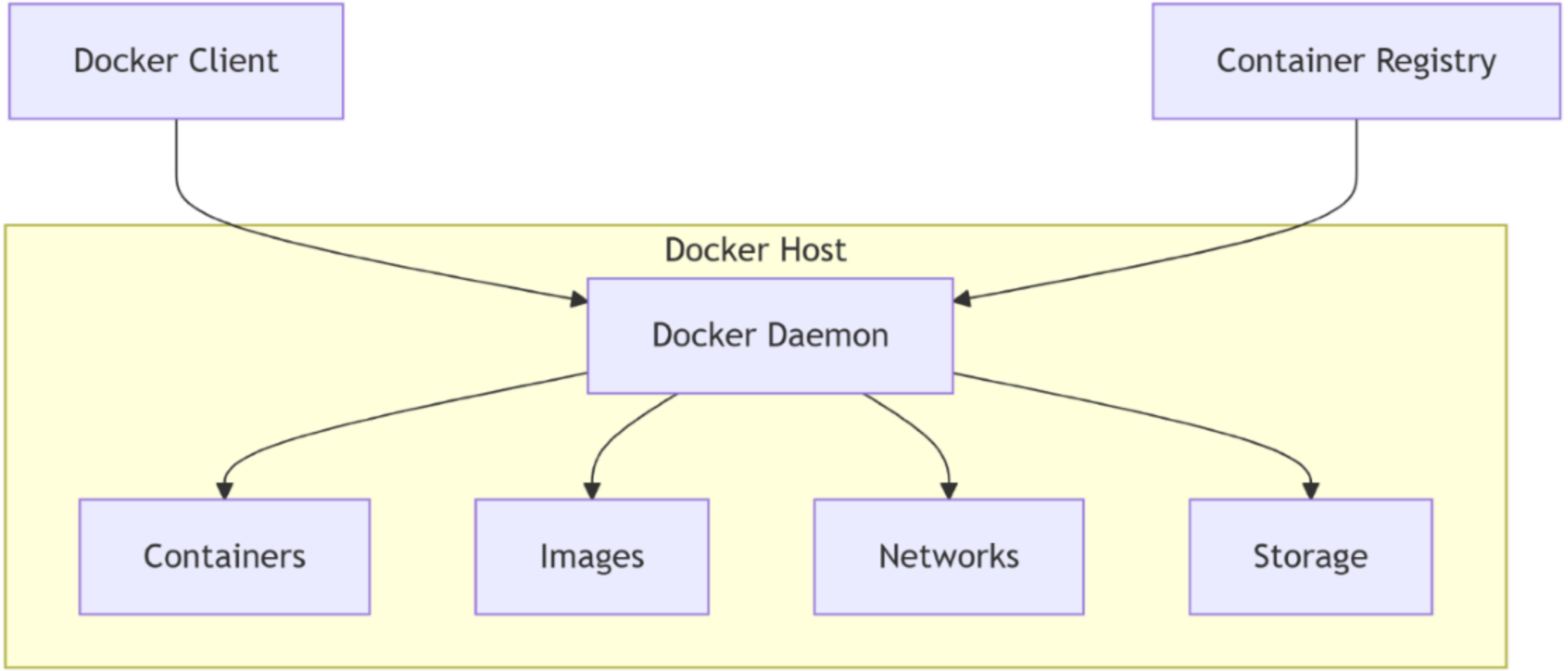
Benefits:

- Isolation - Database runs separately from host system
- Portability - Same environment on Windows/Mac/Linux
- Version control - Precise PostgreSQL versions
- Clean setup - No system-wide installations
- Fast reset - Recreate database in seconds



Docker Architecture

Docker Components



Key Elements:

- **Docker Client** - CLI/REST API Interface
- **Docker Daemon** - Background service that manages containers/images/networks
- **Containerd** - Industry-standard container runtime
- **Container** - Isolated application environment. Runnable instances of images.
- **Images** - Read-only templates (e.g., postgres:16, postgres:17.5, postgres:latest)
- **Registry** - Storage for images (e.g., Docker Hub, Google Container Registry)
- **Networks** - Communication channels between containers

Installation Guide

Windows/Mac

1. Download Docker Desktop (docker.com)
2. Run installer (default settings)
3. Verify: `docker --version in terminal`

Linux

1. Install Docker Engine
2. Verify with: `docker run hello-world`



Running PostgreSQL Containers

Starting a PostgreSQL Container

```
docker run -d --name swen-db \
  -p 5432:5432 \
  -e POSTGRES_PASSWORD=mysecretpassword \
  postgres:16
```

Parameters Explained

- -d - Detached mode (run in background)
- --name - Container name for management
- -p 5432:5432 - Host port : Container port mapping
- -e - Environment variable (sets password)
- postgres:16 - Official image with version tag

Persistent Storage

```
docker run -d --name swen-db \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16
```

Without Volume

- Data lost when container stops

With Volume

- Data persists across container restarts
- Managed by Docker (`docker volume ls`)
- Located at **/var/lib/docker/volumes/**



Container Management Basics

Essential Commands

```
# Start container
docker start swen-db

# Stop container
docker stop swen-db

# Remove container
docker rm swen-db

# Remove image
docker rmi postgres:16
```

Information

```
docker ps -a        # Show all containers
docker images       # List images
docker info         # System-wide information
```



Troubleshooting Common Issues

Connection Problems

Symptoms:

- "Connection refused" errors
- Application can't connect to database

Solutions:

- Verify container running: `docker ps`
- Check port mapping: `docker port swen-db`
- Test internal connection: `docker exec -it swen-db psql -U postgres`
- Examine logs: docker logs swen-db
- Check firewall settings on host machine

Data Persistence Issues

Symptoms:

- Database resets after container restart
- Tables disappear

Solutions:

- Always use volumes:

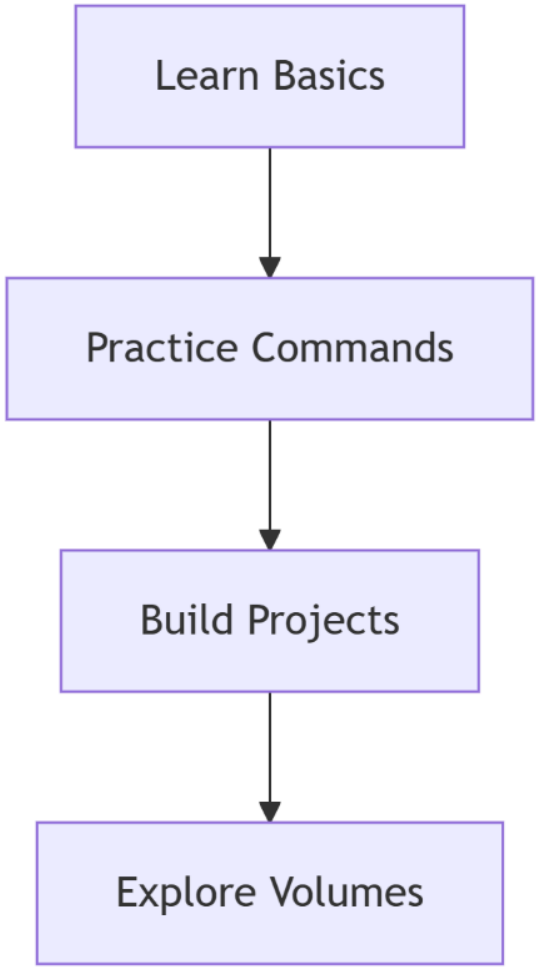
```
-v pgdata:/var/lib/postgresql/data**
```

- Verify volume creation:

```
docker volume ls
docker inspect swen-db | grep Mounts
```

- Avoid using --rm flag during development

Conclusion


Learn Basics
Practice Commands
Build Projects
Explore Volumes

Further Reading

- Official PostgreSQL Docker Image
- Docker Command Line Reference
- PostgreSQL Administration Guide